

Содержание

1	Эволюция языков программирования. Понятие класса, объекта.	3
2	Абстрагирование — определение, пример, реализация на языке.	4
3	Инкапсуляция — определение, пример, реализация на языке.	5
4	Модульность, определение, пример использования.	7
5	Иерархия — определение, виды иерархий, примеры.	7
6	Наследование, виды наследования, примеры.	8
7	Реализация полиморфизма в языках программирования. Виртуальные методы.	8
8	Абстрактные и виртуальные классы.	9
9	Типизация. Статическое и динамическое связывание.	9
10	Статические члены класса. Дружественные члены класса.	9
11	Перегрузка операторов.	10
12	Реализация приведения типов в C++.	10
13	Конструктор копирования.	17
14	Использование паттернов. Пример использования паттерна Фабрика	17
15	Использование паттернов. Пример использования паттерна Одиночка.	17
16	Использование паттернов. Пример использования паттерна Фабричный метод.	18
17	Понятие жизненного цикла разработки ПО. Виды моделей ЖЦ.	22
18	Основные принципы создания хорошего кода.	22
19	Технологии COM, COBRA, .NET. Их идеология и использование. Автоматное программирование.	22

20 Понятие тестирования. Основные типы тестирования.	22
---	-----------

1 Эволюция языков программирования. Понятие класса, объекта.

История современных языков программирования началась ещё в 60-х годах.

1. Первые языки программирования были способом ухода от машинных кодов и ассемблера, данные и программы в них представляли просто куски кода и никак не изолировались друг от друга, из-за чего возникало много проблем.

Представители: ALGOL-58, FORTRAN I.

2. С середины 60-х годов начинают появляться первые процедурные языки, в которых появилась возможность делить программу на подзадачи и использовать ранее созданные процедуры для составления новых программ, однако, контроль над взаимодействием данных и процедур продолжал отсутствовать.

Представители: ALGOL-60, COBOL.

3. К началу 70-х годов появилась возможность изолировать и передавать данные в процедуры. Однако, неудобства, связанные с разработкой больших проектов и большим количеством разработчиков, оставались. Это время расцвета процедурного программирования.

Представители: Pascal, C.

4. Дальше берёт своё начало объектно-ориентированная парадигма, появляется деление программ на модули и объекты, которые взаимодействуют между собой с помощью интерфейсов.

Представители: Симула-2.

5. Дальше уже развивается промышленное и ООП программирование.

Класс — в объектно-ориентированном программировании, модель для создания объектов определённого типа, описывающая их структуру (набор полей и их начальное состояние) и определяющая алгоритмы (функции или методы) для работы с этими объектами.

Иными словами, класс служит средством для введения абстрактных типов данных в программный проект.

Классом принято называть множество всех объектов, обладающих одинаковой структурой и имеющих одинаковую модель поведения.

Объект в программировании — некоторая сущность в цифровом пространстве, обладающая определённым состоянием и поведением, имеющая определённые свойства (атрибуты) и операции над ними (методы). Как правило, при рассмотрении объектов выделяется то, что объекты принадлежат одному или нескольким классам, которые определяют поведение (являются моделью) объекта.

2 Абстрагирование — определение, пример, реализация на языке.

Абстракция данных означает предоставление только важной информации внешнему миру и скрытие их фоновых данных, т. е. Представление необходимой информации в программе без предоставления деталей.

Абстракция данных — это метод программирования (и проектирования), который основан на разделении интерфейса и реализации.

Абстракция нужна для удобства и безопасности пользователя. Вам не нужно знать как работает телевизор, чтобы смотреть его + вам также не нужно знать какие провода надо соединить, чтобы чайник заработал, вам нужно просто знать, где кнопка.

```
#include <iostream>

class CoffeeGrinder
{
private:
    bool checkVoltage(int voltage)
    {
        if (voltage >= 200)
        {
            return true;
        }
        else
        {
            return false;
        }
    }

public:
    void grindCoffee()
    {
        if (checkVoltage(200))
        {
            std::cout << "GRGRGRGRGR" << std::endl;
        }
        else
        {
            std::cout << "BEEP" << std::endl;
        }
    }
};
```

```

int main()
{

    CoffeeGrinder coffeeGrinder;

    coffeeGrinder.grindCoffee();

    return 0;
}

```

3 Инкапсуляция — определение, пример, реализация на языке.

Инкапсуляция — это набор инструментов для управления доступом к данным или методам, которые управляют этими данными.

Инкапсуляция (скрытие) — это процесс отделения друг от друга элементов объекта, определяющих его устройство и поведение; инкапсуляция служит для того, чтобы изолировать контрактные обязательства абстракции от их реализации.

```

#include <iostream>
using namespace std;

class Contact
{
private:
    int mobile_number;
    int home_number;

public:
    Contact()
    {
        mobile_number = 12345678;
        home_number = 87654321;
    }
    void print_numbers()
    {
        cout << "Mobile_number:_ " << mobile_number;
        cout << "_ ,_home_number:_ " << home_number << endl;
    }

    int get_mobile_number()
    {
        return this->mobile_number;
    }
}

```

```

void set_mobile_number(int mobile_number)
{
    if(mobile_number % 10000000 != 0)
    {
        this->mobile_number = mobile_number;
    }
    else
    {
        cout << "Error!" << endl;
    }
}

int get_home_number()
{
    return this->home_number;
}

void set_home_number(int home_number)
{
    if(home_number % 10000000 != 0)
    {
        this->home_number = home_number;
    }
    else
    {
        cout << "Error!" << endl;
    }
}

};

int main()
{
    Contact Tony;
    Tony.print_numbers();

    Tony.set_mobile_number(23456789);
    cout << Tony.get_mobile_number() << endl;

    Tony.set_home_number(34567890);
    cout << Tony.get_home_number() << endl;

    return 0;
}

```

4 Модульность, определение, пример использования.

Модульное программирование — это организация программы как совокупности небольших независимых блоков, называемых модулями.

Использование модульного программирования позволяет упростить тестирование программы и обнаружение ошибок. Аппаратно-зависимые подзадачи могут быть строго отделены от других подзадач, что улучшает мобильность создаваемых программ.

Модуль — это последовательность логически связанных фрагментов, оформленных как отдельная часть программы. Во многих языках (но далеко не обязательно) оформляется в виде отдельного файла с исходным кодом или поименованной непрерывной её части.

При построении модуля используется концепция: «один модуль — одна функция». Таким образом, модуль — это элемент программы, решающий одну самостоятельную задачу.

Например нам нужно написать калькулятор, можно конечно писать сплошным кодом сложение, вычитание, деление и умножение, но при желании разработать калькулятор по типу вольфрама в одном блоке кода в виду сложности не представляется возможным. Тогда нам на помощь приходит модульность, можно разбить саму программу — вольфрам на модули — математические операции и реализовать их как отдельные функции, которые подключаются к общему функционалу самого калькулятора, так мы сможем легко отдельно проверять правильность работы каждой математической операции, легко добавлять функционал к калькулятору, например, добавляя более сложные операции и легко рефакторить код, так как интерфейс взаимодействия остаётся прежним, что позволяет отдельным модулям не зависеть от реализации других модулей. Кроме того такой подход сильно повышает читабельность кода, что немаловажно.

5 Иерархия — определение, виды иерархий, примеры.

Иерархия — упорядочение абстракций, расположение их по уровням.

Иерархия классов есть иерархия по принципу наследования, т.е. типа “это есть разновидность того”. Например, “рабочий есть разновидность персоны”, “автомобиль” есть разновидность “транспортного средства”.

В отличие от этого иерархия объектов — это иерархия по принципу вхождения, т.е. типа “это есть часть того”. Например, “установка — часть завода”, “двигатель” — часть “автомобиля”. Таким образом, объекты нижнего уровня иерархии включаются в объекты более высокого уровня, которые являются для них группой.

В качестве примера первого типа иерархии можно взять иерархию машина → легковой автомобиль/камаз/спорткар/бтр

все они умеют ехать, у всех есть колёса, окна и т.д., но есть и своя специфика, которая добавляется уже в дочерних классах.

В качестве примера второго типа иерархии можно привести иерархию в модели монитора. На нижнем уровне класс световой волны, дальше класс пикселя, дальше класс монитора, состоящего из пикселей.

6 Наследование, виды наследования, примеры.

Наследование (англ. inheritance) — концепция объектно-ориентированного программирования, согласно которой абстрактный тип данных может наследовать данные и функциональность некоторого существующего типа, способствуя повторному использованию компонентов программного обеспечения.

Наследование бывает двух видов:

1) Простое (один предок)

В этом случае наследник наследует методы и, возможно, поля только одного класса родителя.

В качестве примера можно привести родительский класс фигура и его наследники круг, квадрат, треугольник.

2) Множественное (много предков)

В этом случае наследник обладает сразу набором нескольких предков, что позволяет делать код более гибким, однако этот тип наследования очень подвержен ошибкам.

В качестве примера можно привести два родительских класса: оформление и круг, тогда их наследником будет разукрашенный круг, в данном случае два родителя позволят нам работать не просто с фигурой, а уже с стилизованной фигурой — полноценным рисунком.

7 Реализация полиморфизма в языках программирования. Виртуальные методы.

Полиморфизм означает один интерфейс (как перечень объявлений) — много реализаций (определений, связываемых с этими объявлениями).

Виртуальный метод — метод, который может быть переопределён в дочерних классах, он может иметь свою реализацию.

Чисто виртуальный метод — виртуальный метод, не имеющий реализации и запрещающий создавать объекты класса, в котором он определён.

Виртуальные методы нужны для создания интерфейса обращения к общим методам классов иерархии.

8 Абстрактные и виртуальные классы.

Абстрактный класс — класс с одним или несколькими виртуальными методами, объекты которого не могут быть созданы. Служит такой класс только в качестве интерфейса и удобной иерархии, как правило, выступает как самый общий и базовый класс-родитель.

Виртуальным базовым классом называется общий базовый класс для нескольких производных классов, производные классы от которых наследуют только одну копию общего базового класса.

9 Типизация. Статическое и динамическое связывание.

Тип — это коллекция возможных значений.

Типизация — это распознавание языком программирования типа той или иной конкретной переменной.

Типизация бывает:

1) Статическая:

Тип переменной присваивается на этапе компиляции, одной и той же переменной не может быть присвоено несколько типов.

2) Динамическая:

Динамическая типизация — приём, используемый в языках программирования и языках спецификации, при котором переменная связывается с типом в момент присваивания значения, а не в момент объявления переменной. Таким образом, в различных участках программы одна и та же переменная может принимать значения разных типов.

С точки зрения программирования связывание — это процедура установки связи между идентификатором, используемым в коде программы, и его физическим объектом (в общем случае — любым программным компонентом: переменной, процедурой, модулем, приложением и т.д.).

Раннее связывание — установка таких связей до начала выполнения программы. Обычно под этим понимается связывание в процессе компиляции исходных модулей и компоновки исполняемого модуля из объектных.

Позднее связывание — установка связей в процессе выполнения программы.

10 Статические члены класса. Дружественные члены класса.

Статические члены классов — члены, которые создаются в единственном экземпляре для всех объектов этого класса. Доступны они из любого эк-

земпляра класса

Как правило, они используются когда:

- 1) Требуется контроль общего количества объектов класса.
- 2) Требуется одновременный доступ ко всем объектам или их части.
- 3) требуется разделение объектами общих ресурсов.

Дружественные члены класса — члены, которые имеют полный доступ к приватным членам класса, как если бы они были членами самого класса.

Они, как правило, используются, когда вам необходимо использовать какие-то данные в другом классе или функции, а либо предоставлять доступ для всех классов нельзя, либо написание интерфейса доступа к приватным полям класса слишком громоздко и нецелесообразно.

11 Перегрузка операторов.

Перегрузка оператора — способ объявления и реализации оператора таким образом, что он обрабатывает объекты конкретных классов или выполняет некоторые другие действия. При перегрузке оператора в классе вызывается соответствующая операторная функция (operator function), которая выполняет действия, которые касаются данного класса.

Если оператор «перегружен», то его можно использовать в других методах в обычном для него виде.

12 Реализация приведения типов в C++.

Приведение типов в стиле языка C может привести выражение любого типа к любому другому типу данных (исключение это приведение пользовательских типов по значению, если не определены правила их приведения, а также приведение вещественного типа к указателю или наоборот). К примеру, `unsigned int` может быть преобразован к указателю на `double`. Данный метод приведения типов может быть использован в языке C++. Однако, метод приведения типов в стиле языка C не делает проверки типов на совместимость, как это могут сделать `static cast` и `dynamic cast` на этапе компиляции и на этапе выполнения соответственно. При этом все, что умеют `const cast` и `reinterpret cast` данный метод приведения типов делать может.

Общий вид приведения:

`(new type)expr,`

где `new type` — новый тип, к которому приводим, а `expr` — выражение, которое приводится к новому типу.

```
#include <iostream>
//Пустые классы только
//для теста приведения
struct foo{
};
```

```

struct bar{
};
//Наследники bar
struct baz:bar{
};
struct quux:bar{
};

int main()
{
    //Переменные простых типов и указатели на переменные простых типов
    int i = 5;
    double d = 111.222;
    char c = 'a';
    int* pi = &i;
    double* pd = &d;
    const int* cpi = &i;
    void* pv = NULL;
    //Объекты классов
    foo oFoo;
    bar oBar;
    baz oBaz;
    quux oQuux;
    //Указатели на объекты классов
    foo* pFoo = &oFoo;
    bar* pBar = &oBar;
    baz* pBaz = &oBaz;
    quux* pQuux = &oQuux;
    //Приводим явно double к int
    i = (int)d;
    //и наоборот
    d = (double)i;
    //указатель на int к char
    c = (char)pi;
    //char к указателю на void
    pv = (void*)c;
    //указатель на void к указателю на int
    pi = (int*)pv;
    //Снимаем константность const int*
    pi = (int*)cpi;
    //Приводим указатель на объект bar к указателю на объект foo
    //из разных иерархий
    pFoo = (foo*)pBar;
    //Приводим указатель на double к double
    d = (double)pd; //Ошибка!!!
    //А если наоборот?

```

```

    pd = (double*)d; // Ошибка!!!
    //Перемещение из одной иерархии наследования в другую
    pBar = (bar*)pBaz;
    pQuux = (quux*)pBar;
    return 0;
}

```

Оператор приведения `const cast` удаляет или добавляет квалификаторы `const` и `volatile` с исходного типа данных (простые типы, пользовательские типы, указатели, ссылки). Например, был `const int`, а после преобразования стал `int` или наоборот. Квалификаторы `const` и `volatile` называют `cv-квалификаторы` (`cv-qualifiers`). Данные квалификаторы указываются перед именами типов. Как ни трудно догадаться квалификатор `const` задает константность, т.е. защищает переменную от изменения. Квалификатор `volatile` говорит о том, что значение переменной может меняться без явного выполнения присваивания. Это обеспечивает защиту от оптимизации компилятором операций с данной переменной.

Общий вид приведения:

`const cast<new type>(exp)`

```

#include <iostream>
//Снятие константности
void foo(const int* in1, const int& in2)
{
    int *p;
    //Сняли константность и записали 33
    p = const_cast<int*>(in1);
    *p = 33;
    //Сняли константность и записали 55
    const_cast<int&>(in2) = 55;
}
//Добавление константности
void bar(int* in1, int& in2)
{
    const int *p;
    //Добавили константность
    //и пытаемся записать 33
    p = const_cast<const int*>(in1);
    *p = 33; //Ошибка !!!
    //Добавили константность
    //и пытаемся записать 33
    const_cast<const int&>(in2) = 55; //Ошибка!!!
}
//Снятие volatile
void baz(volatile int* in1, volatile int& in2)
{

```

```

    int *p;
    //Сняли volatile и записали 33
    p = const_cast<int*>(in1);
    *p = 33;
    //Сняли volatile и записали 55
    const_cast<int&>(in2) = 55;
}
//Добавление volatile
void quux(int* in1, int& in2)
{
    volatile int *p;
    //Добавили volatile и записали 33
    p = const_cast<volatile int*>(in1);
    *p = 33;
    //Добавили volatile и записали 55
    const_cast<volatile int&>(in2) = 55;
}

int main()
{
    int x=3, y=5;
    std::cout<<x<<"_"<<y<<std::endl;
    //Снимаем константность
    foo(&x,y);
    std::cout<<x<<"_"<<y<<std::endl;
    x=3;
    y=5;
    //Добавляем константность
    bar(&x,y); // Ошибка!!!
    std::cout<<x<<"_"<<y<<std::endl;
    //Снимаем volatile
    baz(&x,y);
    std::cout<<x<<"_"<<y<<std::endl;
    x=3;
    y=5;
    std::cout<<x<<"_"<<y<<std::endl;
    //Добавляем volatile
    quux(&x,y);
    std::cout<<x<<"_"<<y<<std::endl;
    return 0;
}

```

Оператор приведения `static cast` применяется для непалиморфного приведения типов на этапе компиляции программы. Отличие `static cast` от приведения типов в стиле языка C состоит в том, что данный оператор

приведения может отслеживать недопустимые преобразования, такие как приведение указателя к значению или наоборот (unsigned int к указателю на double не приведет), а также приведение указателей и ссылок разных типов считается корректным только, если это приведение вверх или вниз по одной иерархии наследования классов, либо это указатель на void. В случае фиксации отклонения от данных ограничений будет выдана ошибка при компиляции программы. При множественном наследовании static cast может вернуть указатель не на исходный объект, а на его подобъект.

Общий вид приведения:

```
static cast<new type>(exp)
```

```
#include <iostream>
//Пустые классы только
//для теста приведения
struct foo{
};
struct bar{
};
//Наследники bar
struct baz:bar{
};
struct quux:bar{
};

int main()
{
    //Переменные простых типов и указатели на переменные простых типов
    int i = 5;
    double d = 111.222;
    char c = 'a';
    int* pi = &i;
    double* pd = &d;
    const int* cpi = &i;
    void* pv = NULL;
    //Объекты классов
    foo oFoo;
    bar oBar;
    baz oBaz;
    quux oQuux;
    //Указатели на объекты классов
    foo* pFoo = &oFoo;
    bar* pBar = &oBar;
    baz* pBaz = &oBaz;
    quux* pQuux = &oQuux;
    //Приводим явно double к int
    i = static_cast<int>(d);
```

```

//и наоборот
d = static_cast<int>(i);
//указатель на int к char
c = static_cast<char>(pi); //Ошибка!!!
//char к указателю на void
pv = static_cast<void*>(c); //Ошибка!!!
//указатель на void к указателю на int
pi = static_cast<int*>(pv);
//Снимаем константность const int*
pi = static_cast<int*>(cpi); //Ошибка!!!
//Приводим указатель на объект bar к указателю на объект foo
//из разных иерархий
pFoo = static_cast<foo*>(pBar); //Ошибка!!!
//Приводим указатель на double к double
d = static_cast<double>(pd); //Ошибка!!!
//А если наоборот?
pd = static_cast<double*>(d); //Ошибка!!!
//Перемещение из одной иерархии наследования в другую
pBar = static_cast<bar*>(pBaz);
pQuux = static_cast<quux*>(pBar);
return 0;
}

```

Оператор приведения `dynamic cast` применяется для полиморфного приведения типов на этапе выполнения программы (класс считается полиморфным, если в нем есть хотя бы одна виртуальная функция). Если указатель, подлежащий приведению, ссылается на объект результирующего класса или объект класса производный от результирующего то приведение считается успешным. То же самое для ссылок. Если приведение невозможно, то на этапе выполнения программы будет возвращен `NULL`, если приводятся указатели. Если приведение производится над ссылками, то будет сгенерировано исключение `std bad cast`. Несмотря на то, что `dynamic cast` предназначен для приведения полиморфных типов по иерархии наследования, он может быть использован и для обычных непотомных типов вверх по иерархии. В этом случае ошибка будет получена на этапе компиляции. Оператор приведения `dynamic cast` может приводить указатель на полиморфный тип к указателю на `void`, но не может приводить указатель на `void` к другому типу. Способность `dynamic cast` приводить полиморфные типы обеспечивается системой `RTTI` (Run-Time Type Identification), которая позволяет идентифицировать тип объекта в процессе выполнения программы. При множественном наследовании `dynamic cast` может вернуть указатель не на исходный объект, а на его подобъект.

Общий вид приведения:

```
dynamic cast <new type>(exp)
```

```

#include <iostream>
//Пустые классы только

```

```

//для теста приведения
struct foo{
    //Сделали полиморфным
    virtual void do_some(){};
};
struct bar{
    //Сделали полиморфным
    virtual void do_some(){};
};
//Наследники bar
struct baz:bar{
};
struct quux:bar{
};
int main()
{
    //Переменные простых типов и указатели на переменные простых типов
    void* pv = NULL;
    //Объекты классов
    foo oFoo;
    bar oBar;
    baz oBaz;
    quux oQuux;
    //Указатели на объекты классов
    foo* pFoo = &oFoo;
    bar* pBar = &oBar;
    baz* pBaz = &oBaz;
    quux* pQuux = &oQuux;
    //Приводим указатель на объект bar к указателю на объект foo
    //из разных иерархий
    pFoo = dynamic_cast<foo*>(pBar);
    if (pFoo == NULL)
    {
        std::cout<<"FAIL"<<std::endl;//Ошибка на этапе выполнения!!!
    }
    //Приводим указатель на void к указателю на объект bar
    pBar = dynamic_cast<foo*>(pv); //Ошибка на этапе компиляции!!!
    //Приводим указатель на bar к указателю на void
    pv = dynamic_cast<void*>(pBar);
    //Перемещение из одной иерархии наследования в другую
    pBar = dynamic_cast<bar*>(pBaz);
    pQuux = dynamic_cast<quux*>(pBar);
    if (pQuux == NULL)
    {
        std::cout<<"FAIL"<<std::endl;//Ошибка на этапе выполнения!!!
    }
}

```



```

        return 0;
    }

```

13 Конструктор копирования.

Смотри видео у SimpleCode.

<https://www.youtube.com/watch?v=3x9nd6Tm7Pc>

14 Использование паттернов. Пример использования паттерна Фабрика

Используйте паттерн Abstract Factory (абстрактная фабрика) если:

Система должна оставаться независимой как от процесса создания новых объектов, так и от типов порождаемых объектов. Непосредственное использование выражения `new` в коде приложения нежелательно.

Необходимо создавать группы или семейства взаимосвязанных объектов, исключая возможность одновременного использования объектов из разных семейств в одном контексте.

15 Использование паттернов. Пример использования паттерна Одиночка.

Часто в системе могут существовать сущности только в единственном экземпляре, например, система ведения системного журнала сообщений или драйвер дисплея. В таких случаях необходимо уметь создавать единственный экземпляр некоторого типа, предоставлять к нему доступ извне и запрещать создание нескольких экземпляров того же типа.

Паттерн Singleton предоставляет такие возможности.

```

class Singleton
{
private:
    static Singleton * p_instance;
    Singleton() {}
    Singleton(const Singleton&);
    Singleton& operator=(Singleton&);

public:
    static Singleton * getInstance()
    {
        if (!p_instance)
        {
            p_instance = new Singleton();

```

```

        }
        return p_instance;
    }
};

```

Клиенты запрашивают единственный объект класса через статическую функцию-член `getInstance()`, которая при первом запросе динамически выделяет память под этот объект и затем возвращает указатель на этот участок памяти. Впоследствии клиенты должны сами позаботиться об освобождении памяти при помощи оператора `delete`.

Последняя особенность является серьезным недостатком классической реализации шаблона `Singleton`. Так как класс сам контролирует создание единственного объекта, было бы логичным возложить на него ответственность и за разрушение объекта.

Достоинства паттерна `Singleton`

1. Класс сам контролирует процесс создания единственного экземпляра.
2. Паттерн легко адаптировать для создания нужного числа экземпляров.

Недостатки паттерна `Singleton`

1. Трудное использование вложенных `Singleton`
2. Проблематично регулировать время жизни объектов

16 Использование паттернов. Пример использования паттерна `Фабричный метод`.

`Фабричный метод` - это паттерн, который определяет интерфейс для создания объектов некоторого класса, но непосредственное решение о том, объект какого класса создавать происходит в подклассах. То есть паттерн предполагает, что базовый класс делегирует создание объектов классам-наследникам.

Когда надо применять паттерн

1. Когда заранее неизвестно, объекты каких типов необходимо создавать

2. Когда система должна быть независимой от процесса создания новых объектов и расширяемой: в неё можно легко вводить новые классы, объекты которых система должна создавать
3. Когда создание новых объектов необходимо делегировать из базового класса классам наследникам

```
class Program
{
    static void Main(string[] args)
    {
        Developer dev = new PanelDeveloper("ООО_Кирпичи");
        House house2 = dev.Create();

        dev = new WoodDeveloper("Частный_застройщик");
        House house = dev.Create();

        Console.ReadLine();
    }
}

abstract class Developer
{
    public string Name {get; set;}

    public Developer (string n)
    {
        Name = n;
    }

    abstract public House Create();
}
```

```

}

class PanelDeveloper : Developer
{
    public PanelDeveloper(string n) : base(n)
    { }

    public override House Create()
    {
        return new PanelHouse();
    }
}

class WoodDeveloper : Developer
{
    public WoodDeveloper(string n) : base(n)
    { }

    public override House Create()
    {
        return new WoodHouse();
    }
}

abstract class House
{ }

class PanelHouse : House
{
    public PanelHouse()

```

```

        {
            Console.WriteLine("PaneHouse_is_built");
        }
    }

    class WoodHouse : House
    {
        public WoodHouse()
        {
            Console.WriteLine("WoodHouse_is_built");
        }
    }

```

В качестве абстрактного класса Product здесь выступает класс House. Его две конкретные реализации - PanelHouse и WoodHouse представляют типы домов, которые будут строить подрядчики. В качестве абстрактного класса создателя выступает Developer, определяющий абстрактный метод Create(). Этот метод реализуется в классах-наследниках WoodDeveloper и PanelDeveloper. И если в будущем нам потребуется построить дома какого-то другого типа, например, кирпичные, то мы можем с легкостью создать новый класс кирпичных домов, унаследованный от House, и определить класс соответствующего подрядчика. Таким образом, система получится легко расширяемой. Правда, недостатки паттерна тоже очевидны - для каждого нового продукта необходимо создавать свой класс создателя.

**17 Понятие жизненного цикла разработки ПО.
Виды моделей ЖЦ.**

**18 Основные принципы создания хорошего ко-
да.**

- 1) Соблюдать единый стиль кода на протяжении всей программы.
- 2) Не использовать магические числа.
- 3) Использование осмысленных имён.
- 4) Методы по возможности должны быть компактными. Метод решает только 1 конкретную задачу, его функционал должен быть ограничен, он не должен уметь всё.
- 5) Реализуйте проверку всех входящих извне данных на их корректность.
- 6) Наследование используйте разумно (только для шаблона иерархии is-a), по возможности используйте шаблон иерархии "part of".
- 7) Интерфейс и реализация должны быть отделены.

19 Технологии COM, COBRA, .NET. Их идеология и использование. Автоматное программирование.

20 Понятие тестирования. Основные типы тестирования.

Тестирование программного обеспечения (Software Testing) — проверка соответствия реальных и ожидаемых результатов поведения программы, проводимая на конечном

наборе тестов, выбранном определённым образом.

Цель тестирования — проверка соответствия ПО предъявляемым требованиям, обеспечение уверенности в качестве ПО, поиск очевидных ошибок в программном обеспечении, которые должны быть выявлены до того, как их обнаружат пользователи программы.